

Known-Future Covariates Are the Signal: A Patch Transformer with Covariate Query Tokens for Multivariate Operational Load Forecasting

Group ID: 140

Rosela Berberi, Anja Korovesi

Deep Learning (SS26) bonus project, TU Darmstadt

Abstract

We forecast a 336-hour horizon for 96 correlated operational load series from a 168-hour history and 22 covariates. Our central observation is that on this benchmark the covariates are *known in advance* for the whole forecast window, which shifts the problem from extrapolating the target towards learning a covariate-to-load response specific to each unit. We therefore extend the PatchTST backbone with *future covariate query tokens*, a per-series embedding and a pointwise covariate residual path, and contrast it with an LSTM+Attention encoder-decoder that decodes iteratively. On held-out tails of the public training data our submitted model reaches 15.37% WAPE in the validation regime — 59% below the seasonal-mean baseline — and 14.18% in a harder regime reproducing the private split’s 336-step unobserved gap, cutting the naive last-value error by 72%. Ablations attribute almost all of the gain to the covariate pathway rather than to the backbone, and they overturn our headline hypothesis: the recurrent decoder’s collapse across the gap (18.42% $\rightarrow$ 24.09%) is caused by reversible instance normalisation compounding through the rollout, not by iterative decoding as such — removing it recovers 15.92%/14.71% and makes the two families near-equivalent. Finally, a transfer to Jena Climate, where every channel except the calendar is past-only, shows how much of the remaining margin the covariate contract rather than the architecture was buying.

1 Introduction

Multivariate time series forecasting asks for the joint future of several correlated signals given their shared past. This project’s benchmark is an operational instance: 96 anonymised units (`unit_000`–`unit_095`) report an hourly *operational load index* and the task is to predict 336 hours ahead for each. If the index tracks pressure on a unit, as its description suggests, such a forecast is what drives staffing decisions; methodologically it is interesting because the horizon is $2\times$ the conditioning window, so no part of the answer can be read off the recent past by persistence.

Our starting point was an empirical look at the released tables rather than at the model zoo, and it changed the project. The file `validation_input.csv` covers *exactly* the rows of `forecast_index_validation.csv`: all 22 covariates are supplied for the forecast window itself, while the target is withheld. The benchmark is thus not a pure extrapolation task but a *covariate-conditioned* one, and the strongest covariate correlates 0.68 with the target at the same hour, whereas the target’s own autocorrelation has already decayed to 0.26 after six hours. A second measurement shaped the architecture: a per-series ordinary least squares fit on the covariates reaches 17.6% WAPE against 22.1% for a single pooled fit, i.e. the covariate-to-load response is unit-specific even though the static columns (capacity, zone) barely explain anything.

Two consequences follow. A model should consume the future covariates *directly* at the resolution at which they act, which argues for a direct multi-

step decoder over an autoregressive one; and it needs a per-unit degree of freedom. We build both into a PatchTST [16] backbone and compare against the recurrent-attention hybrid from our exposé. Our contributions are **PatchTST-cov**, a patch Transformer whose forecast window enters as patched *future covariate query tokens*, with a learned per-series embedding and a pointwise covariate residual path (Sec. 3); a **gap-aware protocol** that scores every model both immediately after the history and across the 336-step gap separating it from the private test window (Sec. 4.1); an **ablation study** plus a permutation analysis of which covariates the model uses (Sec. 4.3); and a **transfer experiment** on Jena Climate, chosen because it inverts the covariate contract (Sec. 5).

2 Related Work

Recurrent forecasters. LSTMs [8] underpin probabilistic forecasters such as DeepAR [19], which also popularised the per-series embedding we adopt, and additive attention [1] lets a decoder re-read all encoder states. Iterative decoders suffer from *exposure bias* — trained on ground-truth predecessors, they consume their own outputs at test time — for which scheduled sampling [3] is the standard mitigation and the one we use.

Transformers for long-horizon forecasting. After the original Transformer [21], a line of work attacked its quadratic cost and point-wise tokenisation of time series: Informer’s sparse attention [24], Autoformer’s decompo-

sition and auto-correlation [22], FEDformer’s frequency-domain attention [25]. PatchTST [16] instead tokenises *sub-series patches* and processes each channel independently with a shared backbone, preserving local semantics while shortening the sequence by the patch stride; iTransformer [12] inverts the axes and attends over variates. A sceptical counter-line matters for calibration: DLinear [23] showed a single linear layer can match several of these Transformers, and N-BEATS [17], N-HiTS [4] and TiDE [5] reach comparable accuracy with MLPs. We take the lesson seriously: our baselines include the simple references and our ablations include a covariate-free variant, so the gain we claim is attributable to the covariate machinery rather than to attention per se.

Covariate-conditioned models. The Temporal Fusion Transformer [11] is the closest relative of our design: it separates static, past-only and known-future inputs and feeds the latter to the decoder; TiDE [5] does the same with a dense encoder. We borrow the input taxonomy and contribute a minimal way to graft it onto a patch-based backbone, as query tokens on the same token sequence.

Normalisation, regularisation, optimisation. Reversible instance normalisation [10] is the near-default remedy for distribution shift between training and forecast windows; Sec. 4.3 finds it harmful at this horizon, which is one of our main results. We also use spatial dropout [20], AdamW [14] and cosine annealing with warm restarts [13], in PyTorch [18]. Dilated convolutions [2] and deep state-space models [7, 6] are the natural next backbones.

3 Method

3.1 Problem setup and features

For series i and origin t we observe the target history $y_{i,t-L+1:t} \in \mathbb{R}^L$ with $L = 168$, encoder covariates $X_{i,t-L+1:t}^h \in \mathbb{R}^{L \times F_h}$, the known-future covariates $X_{i,t+1:t+H}^f \in \mathbb{R}^{H \times F_f}$ and static features $s_i \in \mathbb{R}^S$, and predict $\hat{y}_{i,t+1:t+H}$.

The 22 released covariates split into six deterministic calendar encodings (hour and day-of-week sine/cosine, a weekend flag, a normalised time trend), 13 exogenous hourly drivers (demand, staffing, queue and network pressure, event load, maintenance, several risk indices) and three static columns (capacity, zone sine/cosine). Ten drivers are missing at random for $\approx 4.5\%$ of rows; we interpolate them within each series and append a binary *missingness indicator* per affected column, so the network can tell an imputed value from an observed one. This yields $F_h = F_f = 29$ and $S = 3$. Covariates are standardised with statistics fitted on the training slice only

and serialised into the checkpoint, so inference reproduces training preprocessing without the training CSV.

The primary metric is $\text{WAPE} = 100 \cdot \sum |y - \hat{y}| / \sum |y|$, whose denominator is constant for a fixed evaluation set. WAPE is therefore a rescaled sum of absolute errors, and the aligned objective is plain L_1 in the original target units [9] — in particular not on per-window normalised targets, which would silently reweight series by their own scale.

3.2 Shared components

RevIN. Each window is normalised by its own mean and standard deviation over time and the prediction is de-normalised with the same statistics [10]. We implement it *statelessly*, passing the statistics explicitly in both directions, so that the conditioning-window statistics survive an autoregressive rollout: caching them on the module — as reference implementations do — lets the decoder’s own call on a single timestep overwrite them with $(\mu, \sigma) = (y_t, 0)$.

Series embedding. A learned vector $e_i \in \mathbb{R}^{16}$ per series is concatenated to s_i [19]. Row 0 of the table is reserved for unseen series, so a checkpoint degrades gracefully rather than failing if the private split introduces a new unit.

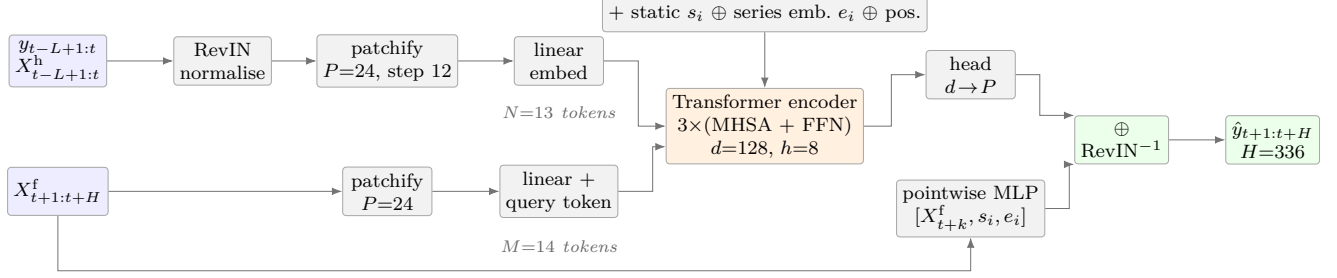
3.3 PatchTST-cov

The backbone follows PatchTST [16]: the RevIN-normalised target history is cut into overlapping patches of length $P = 24$ with stride 12, giving $N = 13$ tokens, each linearly embedded to $d = 128$. Channel independence is preserved in the sense of the original paper — one shared backbone sees a single univariate target channel and every series is an independent sample. Encoder covariates are fused into the history tokens by flattening the corresponding $P \times F_h$ block and projecting it to d .

The extension is on the decoder side (Fig. 1a). We patch the known-future covariates into $M = H/P = 14$ non-overlapping blocks, embed each, add a learned query token, and *append* them to the same token sequence. Self-attention then relates every forecast segment both to the observed history and to the other forecast segments, and a shared head maps each query token’s output to its P target values. The forecast is therefore **direct**: no autoregression inside the horizon, hence no error accumulation.

A patch token compresses 24 hours of covariates into one 128-dimensional vector before the head expands it back to 24 predictions, discarding within-patch resolution. Because the drivers act largely instantaneously, we add a *pointwise residual path*: a two-layer MLP applied independently at each forecast step to $[X_{t+k}^f, s_i, e_i]$, summed onto the patch prediction in normalised space,

(a) **PatchTST-cov** — *direct multi-step decoding*



(b) **LSTM+Attention** — *iterative decoding in 24-step blocks*

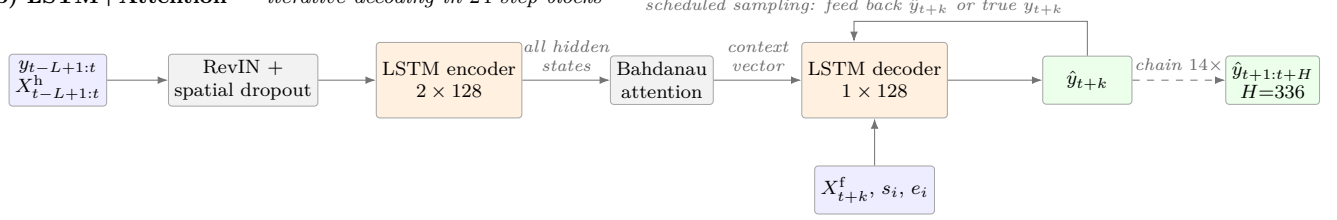


Figure 1: The two architectures. (a) **PatchTST-cov** patches the target history *and* the known-future covariates and lets one Transformer encoder attend over both; the head reads the horizon off the query tokens (a direct 336-step forecast) and a pointwise MLP on X^f restores the resolution patching discards. (b) **LSTM+Attention** decodes 24 steps at a time and chains blocks. Both share the static/series embedding and the raw-scale L_1 objective; the RevIN stage shown here is removed in the final models (Sec. 4.3).

leaving the Transformer to model only the residual cross-step structure. This is the largest departure from our exposé’s plan.

3.4 LSTM+Attention

The recurrent branch (Fig. 1b) is the encoder–decoder from the exposé, adapted to covariates. A two-layer LSTM (128 units) encodes $[\text{RevIN}(y), X^h, s_i, e_i]$ after spatial dropout; a single-layer LSTM decoder unrolls 24 steps, at each step consuming the known covariates of the step being predicted, an additive attention context over all encoder states [1], and either its own previous prediction or the ground truth with probability ϵ . We anneal ϵ linearly from 0.9 to 0.1 across training [3]. Horizons beyond one block are covered by chaining: predictions are appended to the conditioning window together with the known covariates of those steps, and the window is trimmed back to L .

3.5 Ensemble, training and the gap

The two families make structurally different errors, so we blend them after the rollout with a single scalar weight w on PatchTST-cov, chosen by grid search. Which split that search runs on turns out to matter more than the value it returns, and Sec. 4.3 returns to it. Members are trained separately with AdamW (lr = 10^{-3} , weight decay 10^{-4}), gradient clipping at 1.0, batch size 64, cosine annealing with warm restarts [13] and L_1 loss; train-

ing windows are cut with stride 12 (PatchTST-cov) or 24 (LSTM). Predictions are clipped from below at the smallest target seen in training, since the load index is strictly positive.

At private evaluation the input directory provides only the test covariates and the forecast index, and the last *released* target lies 336 steps earlier. Our checkpoint therefore bundles a 504-step tail of the public history (with targets) plus the public validation covariates; `predict.py` rebuilds one continuous per-series timeline and rolls forward to the end of the requested index — two chained 336-step blocks for PatchTST-cov, 28 for the LSTM.

4 Experiments on the benchmark

4.1 Protocol

The public release contains no validation labels, so *every* number we report comes from a chronological split of `train.csv` (96 series $\times$ 4,320 hourly steps, 2023-01-01 to 2023-06-29). The last 672 steps are held out and split in two:

The **validation regime** scores steps 3,649–3,984 with the forecast origin at the end of the fit slice, mirroring the public leaderboard; the **test regime** scores steps 3,985–4,320 from the *same* origin, so the model must first bridge 336 unobserved steps, mirroring the private split. Model selection — early stopping, the blending weight,

Table 1: Benchmark results on the held-out tails of `train.csv`; lower is better, WAPE is the leaderboard’s primary metric, and the test regime starts 336 steps after the conditioning window. * marks the variant selected on validation and shipped; full six-metric tables are in `results/`.

Method	Validation		Test (gap)	
	MAE	WAPE	MAE	WAPE
Naive last value	4.91	52.34	5.34	49.96
Seasonal naive, lag 24	5.37	57.18	4.94	46.26
Seasonal naive, lag 168	5.15	54.88	4.56	42.74
Seasonal mean, dow $\times$ hour	3.50	37.27	3.35	31.38
PatchTST-cov	1.49	15.92	1.66	15.56
PatchTST-cov, no RevIN*	1.47	15.70	1.55	14.47
LSTM+Attention	1.73	18.42	2.57	24.09
LSTM+Attention, no RevIN*	1.49	15.92	1.57	14.71
Blend of the two starred, $w=0.55$	1.44	15.37	1.51	14.18

all hyperparameters — uses the validation regime only; the test regime is scored once per final checkpoint. Our metric code is a copy of the leaderboard’s, but the evaluation *set* is a held-out tail of `train.csv` rather than the hidden validation split, so these values are not directly comparable to leaderboard scores.

4.2 Main results

Table 1 collects the results. Both neural models clear the mandatory naive last-value reference by a wide margin, and both also clear the stronger seasonal-mean baseline. PatchTST-cov reaches 15.70% WAPE in the validation regime against 37.27% for the seasonal mean and 52.34% for the naive baseline, and 14.47% in the gap regime — a 72% error reduction over the naive baseline the project requires us to beat.

Our exposé asked whether direct multi-step output mitigates the error accumulation of an iterative decoder. The first pair of rows says yes, emphatically: with RevIN in place — the configuration we planned — PatchTST-cov is regime-insensitive (15.92% to 15.56%) while the LSTM collapses from 18.42% to 24.09% once it has to chain 28 blocks instead of 14. That is the answer we expected, and it is misleading. The ablation in Sec. 4.3 traces almost all of that collapse to RevIN rather than to iterative decoding, and the second pair of rows confirms it at full budget: without RevIN the same recurrent architecture scores 15.92% and 14.71%, within 0.25 WAPE of the Transformer in both regimes. Decoding strategy was never the dominant factor; a normalisation choice interacting with the rollout was.

Fig. 2 makes the mechanism visible. Fitting a line to MAE against the forecast step, the pre-repair recurrent model drifts upward by +0.26 MAE per 100 steps, while PatchTST-cov, the repaired LSTM and their blend all drift by +0.01 — a factor of 25, and flat to within the noise of the curve. Error accumulation on this benchmark is a property of that one normalisation choice, not of recurrence. Because the two families now make comparable but differently-shaped errors, blending them fi-

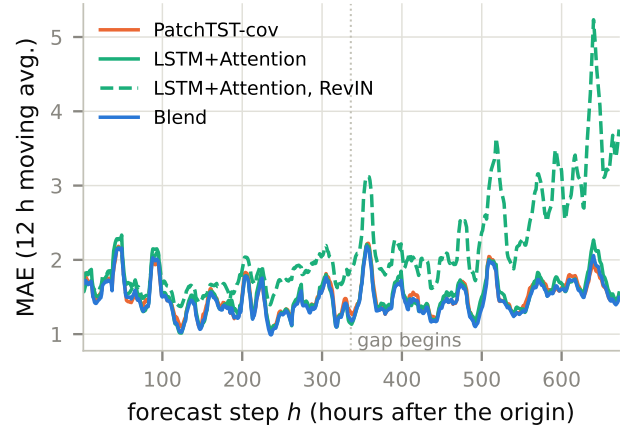


Figure 2: MAE against the forecast step h (12-hour moving average). Both held-out windows share one origin and are contiguous, so $h = 1 \dots 336$ is the validation regime and $h = 337 \dots 672$ the test regime. Only the pre-repair recurrent model (dashed) drifts upward with h ; the two repaired models and their blend are flat and almost indistinguishable. Fitted drift per 100 steps: +0.26 MAE with RevIN versus +0.01 for every model without it.

nally pays: $w = 0.55$ gives 15.37% and 14.18%, better than either member in either regime, with both selection criteria of Sec. 4.3 agreeing on the weight. That blend is our submitted model.

4.3 Ablations and analysis

Table 2 removes one component at a time.

The covariate contract dominates everything else. Removing the future covariate tokens costs over 23 WAPE points, and removing the covariates entirely (vanilla channel-independent PatchTST) over 30; the recurrent model loses 18.9 the same way. Every other edit in the table moves the metric by less than two points. This is the empirical core of the paper: on this benchmark the attention backbone is not what produces the accuracy — the availability of future covariates is — and the architecture’s job is to consume them at the right resolution. Consistent with that, the pointwise residual path, whose only purpose is to preserve within-patch resolution, is the most valuable remaining component, and patch length matters little once it is present. Attention is the recurrent branch’s equivalent load-bearing part.

Two components did not survive contact with the data. The per-series embedding, which our per-unit OLS diagnostic predicted would matter, moves WAPE by +0.34 on validation and is within noise for the LSTM. The honest reading is that the covariate pathway already carries the unit-specific response the pooled linear fit

Table 2: Ablations. Every row — including the two references — uses the same reduced budget (stride 96, 8/5 epochs), so each Δ is attributable to the ablated component rather than to a different amount of training; absolute values are therefore worse than Table 1. Positive Δ is worse. “No future cov. tokens” necessarily also removes the pointwise path and reverts to PatchTST’s flatten head.

Configuration	Validation		Test (336h gap)	
	WAPE	Δ	WAPE	Δ
PatchTST-cov (ref.)	16.92		16.49	
MSE instead of L_1	17.78	+0.85	17.92	+1.43
no RevIN	16.27	-0.65	15.26	-1.23
no covariates (vanilla)	47.70	+30.78	37.41	+20.92
no future cov. tokens	40.19	+23.27	42.02	+25.53
no pointwise path	18.52	+1.60	18.06	+1.58
no series embedding	17.27	+0.34	16.63	+0.14
patch 48, step 24	17.69	+0.77	17.53	+1.04
patch 8, step 4	17.30	+0.37	16.27	-0.21
LSTM+Attention (ref.)	21.65		30.60	
free running only	21.13	-0.52	30.25	-0.35
no RevIN	20.32	-1.33	19.57	-11.03
no attention	32.68	+11.03	36.52	+5.93
no covariates	40.52	+18.88	40.94	+10.35
no series embedding	21.52	-0.12	32.21	+1.61
teacher forcing only	22.60	+0.95	34.29	+3.69

could not: once a unit’s drivers are read at full resolution, its identity adds little.

RevIN is worse than that — it actively hurts, by -0.65 for PatchTST-cov and by -1.33 on validation and -11.03 across the gap for the LSTM (a negative Δ means removing the component helps). The asymmetry is the clue. RevIN re-injects the conditioning window’s own mean and standard deviation into the prediction; over 336–672 steps last week’s level is a poor estimate of next fortnight’s, so the transform adds variance rather than removing shift. For the *iterative* decoder it is worse than that: every chained block re-estimates those statistics *from the block it just predicted*, so a level error is fed back into the normaliser and amplified at each of the 28 hops. That is the mechanism behind the collapse we first attributed to iterative decoding. Because the sign is the same on validation, we could act on it without consulting the test split — we re-trained both families without RevIN at the full budget (Table 1), and both improved, the LSTM by 9.4 WAPE in the gap regime. This is the one place where the ablation study changed the submitted model rather than merely explaining it.

Scheduled sampling helps, mostly where it should. Teacher forcing only ($\epsilon = 1$) is worse in both regimes and three times more so across the gap — the classic exposure-bias failure, visible mainly where more blocks are chained. Free running only ($\epsilon = 0$) is *not* worse than the annealed schedule here, so on this dataset the annealing buys optimisation stability rather than final accuracy. Replacing L_1 with MSE costs accuracy under WAPE, as the metric-alignment argument of Sec. 3 predicts.

Tuning the blend weight needs a gap-matched criterion. A weight tuned on the validation regime is tuned on a regime the model never runs in, because the public leaderboard has no gap. We therefore also grid-search w on a *gap-matched* criterion: the same validation labels, but with the forecast origin moved back one window so the members must bridge 336 unobserved steps first (15.40 % for the blend). With the RevIN-free members the two criteria happen to agree on $w = 0.55$, so the distinction costs us nothing here — but it did matter before the repair, when the val-tuned weight of 0.80 improved validation by 0.23 and lost 0.26 in the gap regime. Both grids are recorded in `results/ensemble.json`.

What the model uses. Shuffling one covariate at a time across the forecast window gives a concentrated ranking: from a base of 15.70 %, destroying `queue_pressure_forecast` alone costs +14.1 WAPE and `network_pressure_forecast` +7.6, then `maintenance_known` (+2.9) and `demand_forecast` (+2.0); nothing else reaches +1.7 and the calendar encodings are negligible. Two of the 22 inputs carry most of the accuracy. The values are a lower bound per channel, since the covariates are mutually correlated; the full ranking is in `figures/permutation_importance.pdf`.

5 Additional dataset: Jena Climate

Dataset and use case. The Jena Climate dataset [15] records 14 atmospheric variables — air pressure, air and potential temperature, dew point, relative humidity, three vapour-pressure channels, specific humidity, water-vapour concentration, air density, wind speed, maximum wind speed and wind direction — every 10 minutes at the Max Planck Institute for Biogeochemistry weather station, from 2009-01-01 to 2017-01-01 (420,551 rows). The forecasting use case is short-to-medium range local weather nowcasting — the next two weeks of temperature, and of pressure as a second target, from the recent multivariate state.

We chose it because it *inverts* the benchmark’s covariate contract: nothing except the calendar is known in advance, so it tests whether our gains come from modelling or from unusually generous inputs. It is also harder as a multivariate problem — the channels are non-linearly coupled (dew point and vapour pressure are functions of temperature and humidity) and the target has strong diurnal *and* annual seasonality, the latter longer than any window we condition on.

What had to change. Exactly one abstraction: a *past-only* covariate group. Encoder features become [known-future $\cup$ past-only] while decoder features are the known-future subset alone, so $F_h = 24 > F_f = 8$;

Table 3: Jena Climate, hourly, $L = 168$, $H = 336$, rolling-origin evaluation. Temperature crosses zero, so MAE and RMSE replace WAPE. Both targets use the same architecture and optimiser settings as the benchmark, with a reduced epoch budget and no static or missingness features; the calendar encodings are the only known-future covariates.

Method (MAE)	temperature [$^{\circ}$ C]		pressure [mbar]	
	val	test	val	test
Naive last value	3.79	5.96	8.04	7.88
Seasonal naive, lag 24	3.53	5.00	8.59	8.09
Seasonal mean	5.56	6.69	6.73	6.49
PatchTST-cov (ours)	3.04	3.76	7.23	6.42
LSTM+Attention (ours)	2.94	3.79	8.19	7.00

both architectures read the two widths from the specification and no layer was rewritten. When the iterative decoder appends a predicted block to its window it cannot know the past-only channels there, so they are filled by persistence — never by their true future values, which would leak the answer. Preprocessing is dataset-specific: hourly means to match the benchmark’s rate, repair of the station’s -9999 wind sentinel, Cartesian wind components in place of the circular direction column, and day-of-year sine/cosine for the annual cycle. The split is chronological 70/10/20; because the held-out slices span thousands of steps we use the standard rolling-origin protocol (one origin every 336 steps, $L = 168$, $H = 336$). Temperature crosses zero, so we report MAE and RMSE rather than WAPE.

Does it transfer? Only partly, and the pattern of where it fails is the useful result (Table 3). On *temperature* the architecture transfers cleanly: both models beat every baseline on both splits, cutting test MAE from 5.00 (the best baseline, seasonal naive) to 3.76, a 25 % reduction, with no tuning beyond the feature specification. On *pressure* it essentially fails: PatchTST-cov is worse than the seasonal-mean baseline on validation (7.23 vs 6.73) and only ties it on test (6.42 vs 6.49), and the recurrent model loses on both.

The contrast is what our thesis predicts. Nothing here is known in advance except the calendar, so the query tokens and the pointwise path have almost no input and the model falls back on target history plus calendar — precisely the regime in which linear baselines are competitive [23]. What is left of the advantage depends on how much of the target the history explains: temperature has a diurnal cycle the encoder can exploit, whereas pressure is driven by synoptic systems invisible in a 168-hour window and whose long-run mean the seasonal baseline already captures. The two decoders are again within 0.03 MAE on temperature, a third sign that the direct/iterative choice is not what separates models here. The backbone, recipe and past-only abstraction all transfer unchanged; the *size* of the win moves with the covariate contract, not the architecture.

6 Conclusion

We set out to compare a recurrent-attention hybrid against a patch Transformer, and the most useful results were not about which backbone wins. Because the benchmark supplies all 22 covariates for the forecast window, the task is a covariate-conditioned regression rather than extrapolation: feeding that window in as patched query tokens and preserving its resolution with a pointwise residual path account for essentially the whole improvement, while the per-series embedding our own diagnostic motivated proved worth almost nothing. The second result is a correction. We measured the large gap-regime collapse we expected from an iterative decoder and would have reported it as evidence for direct decoding; the ablation showed the cause was RevIN’s statistics compounding through the rollout, and removing it left the two decoders within 0.25 WAPE of each other. Blending them then helps in both regimes, and our submitted model reaches 14.18 % WAPE across the gap, a 72 % reduction over the naive baseline. Both findings were only visible because every configuration was scored in a regime matching deployment rather than the one the leaderboard shows.

Limitations. Selection rests on a single 336-step validation window of 96 series with no seed repeats. Rescoring identical checkpoints under a different CPU thread count moves WAPE by up to 0.07 (0.14 where a chained rollout amplifies float-summation order), so differences below ≈ 0.1 in Table 2 are not real. Training ran on CPU, which capped the search and the epoch budget. The gap handling assumes the private window is contiguous with the public history. Finally the model is deterministic, and its spike-dominated residual ([figures/forecast_examples.pdf](#)) is where a quantile head [19] should help.

Future work. Beyond a probabilistic head: cross-series attention over the 96 units, whose correlations our channel-independent treatment ignores, and a comparison against a strong MLP covariate model such as TiDE [5] to test how much attention actually adds.

Contributions

Rosela Berberi — data analysis establishing the covariate contract and the per-series diagnostic; feature pipeline; PatchTST-cov; training and evaluation harness including the gap-aware protocol; inference and submission packaging. **Anja Korovesi** — LSTM+Attention including the RevIN statefulness fix, scheduled sampling and the chained rollout; baselines and metrics; ablation study and permutation analysis; Jena Climate selection, preprocessing and the past-only abstraction; figures. Experiments were designed and the report written jointly.

References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.
- [2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [3] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.
- [4] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. NHITS: Neural hierarchical interpolation for time series forecasting. In *AAAI Conference on Artificial Intelligence*, 2023.
- [5] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with TiDE: Time-series dense encoder. *Transactions on Machine Learning Research*, 2023.
- [6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024.
- [7] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- [8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [9] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- [10] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.
- [11] Bryan Lim, Serkan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- [12] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.
- [13] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [15] Max Planck Institute for Biogeochemistry. Jena climate dataset: Weather station data, 2009–2016. <https://www.bgc-jena.mpg.de/wetter/>, 2016. Accessed August 2026.
- [16] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- [17] Boris N. Oreshkin, Dmitri Carpo, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020.
- [18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 2019.
- [19] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.
- [20] Jonathan Tompson, Ross Goroshin, Arjun Jain, Yann LeCun, and Christoph Bregler. Efficient object localization using convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2015.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

- [22] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.
- [23] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.
- [24] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.
- [25] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, 2022.